

Research Statement

Cross-Layer Co-Design for Next-Generation Datacenter Systems

Inho Choi

1 Overview

I am a systems and networking researcher working at the intersection of academic rigor and industrial practice. Through **cross-layer co-design** across software, hardware, and artificial intelligence, I tackle critical systems challenges and build the next-generation datacenter as a single integrated system, where functionality is placed where it most belongs.

Modern datacenters face a fundamental challenge: serving an ever-expanding spectrum of latency-critical and data-intensive workloads, ranging from microservices and real-time analytics to large-scale AI training and inference. These workloads demand both microsecond-scale tail latency and massive throughput at unprecedented scale. In response, datacenter network fabrics have scaled exponentially, with hyperscalers now entering the era of terabit networking. However, host-side compute has not kept pace. With the end of Moore’s law and Dennard scaling, single-thread CPU performance has plateaued, while the volume and complexity of data each host must process continue to grow. As a result, the host, running applications and systems software, is emerging as the dominant performance bottleneck in modern datacenters.

Meanwhile, two new opportunities are reshaping the datacenter landscape. First, **specialized hardware accelerators** (e.g., SmartNICs, programmable switches, FPGAs, GPUs) are becoming widely available across the datacenter. Each is tailored to a specific class of computation, offering much higher performance and energy efficiency than general-purpose processors. Second, **machine learning** is increasingly being integrated into core system design. Recent advances have made ML models fast and accurate enough to drive real-time decisions such as OS parameter tuning, resource allocation, and scheduling, where hand-crafted heuristics fall short. In parallel, advances in generative AI now allow synthesis of realistic system data at scale, addressing the long-standing scarcity of representative traces in tasks such as system analysis, anomaly detection, and model training.

Approach and directions. Given that the emerging host bottleneck stems from growing compute demands and saturating CPU performance scaling, optimizing the software layer independently faces fundamental limits. This motivates **cross-layer co-design** of modern datacenter systems: carefully distributing functionality across software, hardware, and machine learning. My research applies this methodology along three connected directions. The first, **co-designing distributed systems with programmable network hardware**, has been the main focus of my doctoral research, where I have introduced programmable network hardware into distributed systems [1, 2, 3, 4]. Building on this foundation, the second, **co-designing operating systems with machine learning**, is my ongoing research, where I am bringing ML into the OS as a first-class system component for real-time adaptive optimization [5]. The third, **co-designing AI infrastructure with hardware accelerators**, is my future direction, where I plan to build systems that coordinate heterogeneous HW accelerators for AI workloads [6]. Together, these directions form my long-term vision: treating the entire datacenter as a single integrated system where functionality is placed where it most belongs, a design principle that moves next-generation datacenters beyond the limits of single-layer

optimization.

Impact and recognition. My research is grounded in both academic rigor and the realities of industry datacenters at scale. Through three research internships at Microsoft Research (MSR) and AMD, I have built systems that target the bottlenecks faced by large-scale production datacenters, with a consistent emphasis on designs that are both technically novel and practically deployable. My work has been recognized by both industry and the research community for not only overcoming the practical deployability and performance limitations of prior state-of-the-art designs but also inspiring new design paradigms for datacenters. The resulting systems have appeared as first-author publications at top networking systems venues, including NSDI and SIGCOMM, and have been demonstrated on MSR datacenters. This impact continues through my ongoing close collaboration with MSR, where I am further advancing my research at the intersection of cutting-edge systems and production datacenters.

2 Co-Designing Distributed Systems with Programmable Network Hardware

Traditionally, distributed systems have treated the network as a best-effort fabric that simply forwards and routes packets between hosts. With programmable network hardware, the network can execute custom logic and serve as a primitive for fundamentally new distributed protocols, rather than just a faster substrate for existing ones. My research co-design distributed systems with network primitives to address three critical datacenter challenges: distributed consensus (Hydra [1]), network load balancing (ConWeave [2]), and server load balancing (Capybara [3, 4]).

2.1 Hydra: Replacing Consensus with Scalable Network Ordering (NSDI '23) [1]

Distributed servers have traditionally relied on consensus algorithms (e.g., Paxos) to maintain a consistent request order, but their expensive per-request coordination has become a key bottleneck in high-performance datacenters. Network ordering systems (e.g., NOPaxos) instead order requests by serializing all requests through a single programmable switch (sequencer) within the network, removing this coordination overhead. However, this single-sequencer design caps system scalability, creates network load imbalance, and makes switch failover prohibitively slow, fundamentally blocking practical adoption.

Hydra is the first work to make network ordering practically deployable at scale. It distributes ordering across *multiple* programmable switches that operate in parallel, with a novel cross-switch ordering scheme that lets receivers merge per-switch sequences without any coordination among the switches themselves. Hydra effectively addresses all three limitations of the single-sequencer design, delivering order-of-magnitude improvements over prior network ordering systems in practical deployments.

2.2 ConWeave: Network Load Balancing with In-network Reordering Support for RDMA (SIGCOMM '23) [2]

In modern datacenters, Remote Direct Memory Access (RDMA) powers the most performance-critical workloads, from large-scale AI training to distributed storage. Yet RDMA's strict in-order delivery requirement creates a fundamental load balancing problem: balancing load across multiple network paths inevitably causes packets to arrive out of order, and RDMA hardware treats any out-of-order arrival as congestion and immediately slows the sender. This forces datacenters into a difficult trade-off. Coarse-grained load balancing (e.g., per-flow) cannot evenly distribute RDMA traffic,

while finer-grained schemes (e.g., per-packet) inevitably cause out-of-order arrivals that severely degrade RDMA performance. The trade-off has been a major barrier to fully exploiting RDMA’s capacity at scale.

ConWeave is the first work to break this trade-off by performing fine-grained rerouting while restoring packet order *inside the network itself*, before packets reach the destination server. When congestion arises, the sender-side switch reroutes traffic onto a less congested path; the receiver-side switch then uses hardware features of modern programmable switches to temporarily hold and reorder the diverted packets, delivering them in the original order. ConWeave requires no changes to RDMA hardware and applications, removing a major practical barrier to practical deployments, while significantly outperforming existing load balancing schemes.

2.3 Capybara: Dynamic Server Load Balancing with TCP Migration (APSys '23, SIGCOMM '26) [3, 4]

In datacenters, Layer-4 (L4) load balancers distribute client TCP connections across servers. Because TCP is stateful, each connection must remain pinned to its original server, and L4 load balancers cannot rebalance once connections are established. Due to skewed workload across connections, this static pinning fails to fully prevent load imbalance across servers, causing inefficiency in production datacenters.

Capybara is a novel L4 load balancing architecture built on microsecond-scale *live migration* of established TCP connections, enabling dynamic rebalancing of workloads in response to real-time conditions. When the load balancer detects an overloaded server, it can move some of its connections to underutilized servers in tens of microseconds. Capybara achieves this through a tight co-design between a programmable network switch that redirects connection traffic according to migrations and a host TCP stack that can transfer connection states at microsecond speed. Capybara effectively addresses the limitation of static pinning, delivering order-of-magnitude performance improvements over existing L4 load balancers under realistic workloads.

3 Ongoing and Future Research Directions

Building on this foundation, I am extending cross-layer co-design in two further directions: into operating systems and AI infrastructure layer.

3.1 Co-Designing Operating Systems with Machine Learning (Ongoing)

OS performance tuning is one of the most critical yet difficult problems in datacenter systems, with Google reporting that the OS consumes about 20% of all CPU cycles in its datacenters. Thus, major cloud providers invest heavily in dedicated OS optimization teams that tune dozens of parameters across networking, scheduling, and memory subsystems, which interact in non-obvious ways with optimal settings shifting across workloads and hardware. Modern datacenter workloads have intensified this challenge with their microsecond-scale performance demands, pushing tuning well beyond what fixed rules or human-crafted heuristics can handle. While recent ML advances could address tuning challenges beyond what heuristics can express, the OS itself is not designed to host ML as a system component. Conventional OS architectures lack a uniform tuning interface across subsystems, expose no observation points for high-quality training data, and offer no inference path that meets tight latency budgets of dataplane, leaving ML as an external optimizer confined to narrow tasks.

Autokernel [5] proposes a new design paradigm for OS architecture that treats machine learning as

a first-class system component. This ML-native architecture provides a uniform tuning interface across networking, scheduling, and memory subsystems, observation points wired into the dataplane to generate training data, and a hybrid inference path that combines offline lookup tables for well-understood regimes with lightweight ML models for unseen workloads, all within the microsecond-scale dataplane I/O processing. Autokernel co-designs the OS with ML, making continuous workload-adaptive optimization a deployable OS property that future ML policies can build on without re-architecting the integration. In preliminary evaluation, Autokernel sustains microsecond-scale tail latency under dynamic workloads where a statically-tuned baseline incurs millisecond-scale queuing delays. Full-system development is ongoing in close collaboration with Microsoft Research.

While Autokernel establishes how ML should integrate with the OS, the bottlenecks ML must adapt to remain poorly characterized in community. In a complementary effort, I am empirically characterizing intra-host network bottlenecks across modern datacenter hardware, such as PCIe complex and I/O subsystems.

Building on these foundations, I plan to advance ML-native systems along three forward-looking research thrusts:

- **Safe and verifiable learned OS policies.** Operators need confidence that learned decisions do not violate system-level safety, performance, or liveness guarantees. I plan to develop a safety framework for ML-native OSes where developers declare constraints their ML policy must satisfy (e.g., output ranges, fairness) and fallback actions when constraints are violated (e.g., safe baseline, retraining). These declarations compile into lightweight OS monitors that enforce the rules at runtime. Instead of each ML-driven system reinventing safety checks from scratch, a shared framework lets operators deploy ML policies with the same trust they have in conventional system code.
- **Generative AI for ML-native systems.** Training-data scarcity remains a persistent obstacle for ML-native systems, since production traces are difficult to obtain and rarely shared. Generative AI can synthesize realistic system traces, but faces a fundamental tradeoff between expressiveness (capturing diverse patterns) and scalability (learning efficiently at scale). I plan to resolve this tradeoff by capturing complex patterns through compact summary variables, and by learning active feedback from self-tuning systems instead of passive traces alone. This framework will produce realistic synthetic traces (packet flows, syscall sequences, scheduling decisions, request patterns) usable for training, evaluation, and what-if analysis in ML systems research.
- **Extending ML-native design to other OS domains.** ML-native design is not specific to traditional host OSes. I plan to apply this principle to GPU host OSes (where memory-allocator, scheduler, and interconnect tuning shape AI workload performance) and embedded/robotics OSes (where adaptive resource management is critical under tight real-time constraints). These extensions will turn ML-native OS into a paradigm that applies to broad operating system domains.

3.2 Co-Designing AI Infrastructure with Hardware Accelerators (Future)

AI workloads such as LLM training and inference, and emerging agentic systems are fundamentally different from the workloads that shaped previous decades of datacenter design. They are often limited by data movement rather than raw compute, span heterogeneous memory tiers (HBM, DRAM, CXL), and execute across thousands of GPUs coordinated by specialized fabrics (NVLink, InfiniBand). Together, they expose a level of asymmetry between accelerator compute and host I/O that existing system stacks were not designed to handle. My exposure to this design space

began with RecoNIC [6], an industrial collaboration with AMD that gave me direct experience with SmartNIC platforms and with the host–device interaction patterns that dominate AI infrastructure performance.

RecoNIC is a hardware–software co-designed platform for RDMA-enabled compute offloading on FPGA-based SmartNICs. The motivation arose from a fundamental inefficiency in conventional FPGA deployments: data arriving from the network is first DMA’d into host memory, then copied over PCIe to the FPGA for processing, and finally copied back to host memory before transmission. These extra hops cause significant performance overhead. RecoNIC addresses this by integrating an RDMA engine and programmable compute-logic modules directly into the SmartNIC’s hardware shell, allowing network data to be placed directly into device memory and processed in place by user-defined accelerators.

More broadly, this work taught me that careful coordination across different system layers is essential for translating advances in individual components like FPGA-based SmartNICs into system-level benefits. Inspired by this experience, my future research aims to treat AI infrastructure as a single co-designed system rather than a collection of independently optimized components. Three connected directions will drive this research: cross-layer dataplanes that orchestrate data movement across accelerators and memory tiers, ML-driven policies that adapt placement and scheduling to workload dynamics, and programmable network HW that offload critical-path operations from host CPUs. This vision is a natural synthesis of my prior work, including HW-SW co-design (Hydra, Cappybara), ML-native OS (Autokernel), and AI acceleration (RecoNIC), applied to the AI infrastructure that will define the next-generation datacenter design.

4 Conclusion

Datacenter systems are evolving rapidly: workloads driven by AI, microsecond-scale services, and agentic systems demand more than incremental tuning, while specialized hardware and machine learning expand the design space across the stack. My research pursues **cross-layer co-design**: deliberately redistributing system functionality across hardware, software, and machine learning. I carry this methodology across the datacenter stack: from the distributed systems layer, into the host system itself, and toward the broader AI infrastructure layer. Together, these directions are united by a single goal: redrawing the boundaries between hardware, software, and AI to meet the demands of next-generation datacenter systems.

References

- [1] **Inho Choi**, Ellis Michael, Yunfan Li, Dan Ports, and Jialin Li. *Hydra: Serialization-Free Network Ordering for Strongly Consistent Distributed Applications*. In Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23), 2023.
- [2] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, **Inho Choi**, Jialin Li, and Mun Choon Chan. *Network Load Balancing with In-network Reordering Support for RDMA*. In Proceedings of the 2023 ACM SIGCOMM Conference (SIGCOMM '23), 2023.
- [3] **Inho Choi**, Nimish Wadekar, Raj Joshi, Joshua Fried, Dan R. K. Ports, Irene Zhang, and Jialin Li. *Cappybara: μ Second-scale Live TCP Migration*. In Proceedings of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '23), 2023.

- [4] **Inho Choi**, Nimish Wadekar, Guangda Sun, Raj Joshi, Joshua Fried, Omar S. Navarro Leija, Dan Ports, Irene Zhang, and Jialin Li. *Capybara: Dynamic Load Balancing with Microsecond-Scale TCP Migration*. In Proceedings of the 2026 ACM SIGCOMM Conference (SIGCOMM '26), 2026.
- [5] **Inho Choi**, Anand Bonde, Jing Liu, Joshua Fried, Irene Zhang, and Jialin Li. *ML-native Dataplane Operating Systems*. In Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25), 2025.
- [6] Guanwen Zhong, Aditya Kolekar, Burin Amornpaisannon, **Inho Choi**, Haris Javaid, and Mario Baldi. *A Primer on RecoNIC: RDMA-enabled Compute Offloading on SmartNIC*. arXiv:2312.06207, December 2023.